HeatWise

// PRODUCT DOCUMENTATION · v1.0

User Flow, Features & Tech Spec

An AI-powered urban heat island mitigation platform. Scan your rooftop or terrace, get real-time green cover recommendations, connect with verified installers, and track your carbon footprint — all in one app.

Next.js 16 · Prisma + PostgreSQL · Python ML Serving · Capacitor Android · NextAuth OTP · Three.js AR · Open-Meteo Weather

27 APP SCREENS	80+ API ENDPOINTS	55+ DATA MODELS
AR LIVE MEASUREMENT	ML SPECIES ENGINE	CO_2 CARBON TRACKING

What is HeatWise?

HeatWise is an AI-powered mobile application that helps urban residents and property owners combat the heat island effect by designing, installing, and monitoring green cover on rooftops, terraces, and balconies. Users scan their space with a phone camera, receive ML-generated plant species recommendations, connect with verified local installers, and track the resulting carbon offset — all within a single seamless journey.

> **Core Value Proposition**
>
> Every 10 m² of green cover installed drops surface temperature by ~0.67°C, saves 5–15% in air conditioning energy, and sequesters ~0.21 kg CO_2 per year. HeatWise makes this measurable, actionable, and verifiable at scale.

All App Screens (27 total)

ONBOARDING & AUTH

🌇 Splash
3D mega-city scene with animated buildings and heat shimmer. Entry point on cold launch.

👋 Onboarding
3-step intro carousel explaining the app's value. Skip or proceed to auth.

📱 Phone Login
Phone number entry with country code selector. Initiates OTP flow.

🔢 OTP Verify
6-digit code entry with countdown timer and resend option.

👤 Profile Setup
4-step onboarding: name/age, city auto-detect, gardening interests, space type and size.

⚙️ Complete Profile
Extended profile completion for returning users who skipped earlier fields.

CORE WORKFLOW

🏠 Home Dashboard
City Cooling Index, metric cards, carbon card, project list, explore shortcuts.

+ Create Project
Project name, surface type, primary goal (cooling / food / aesthetics).

📏 Manual Measure
Width, length, floor level sliders with real-time area calculation.

📷 Photo Capture
Camera shutter or gallery import. Gallery mode supports letterbox-corrected tap.

🔭 Live AR Measure
Real-time perspective-correct corner placement using device orientation sensors.

🖼️ Photo AR Overlay
Measurement overlay drawn on a captured photo with gallery angle picker.

🌤️ Environment
Auto-detect via geolocation + weather API or manual sun/wind/temp sliders.

🧠 Analysis

🌿 Results
Ranked species cards with sun / wind / cooling preference filters.

Neural-brain animation while the recommendation engine runs (ML + rules).

POST-RECOMMENDATION & FEATURES

🗺️ Garden Layout
Zone-based spatial layout (north wall, perimeter, center) with interactive plan.

✨ Before/After
Side-by-side 3D garden visualization. Swipeable with cooling delta overlay.

📄 Report
Detailed recommendation dossier: species info, cost breakdown, installation guide.

🔧 Install
Installer search, assignment, quote request and acceptance flow.

✅ Install Done
Confirmation screen after verified installation. Triggers follow-up schedule.

💾 Saved
Bookmarked recommendations and favorited layouts.

🗺️ City Heat Map
Real-time ambient temperature overlaid on a holographic 3D city globe.

🌱 Species Library
Searchable catalog of all supported plant species with trait filters.

🌍 Carbon Track
Footprint setup (4 inputs) and dashboard with garden offset calculation.

📊 Impact Dashboard
Aggregate CO_2, temperature drop, water, and species metrics with animated counters.

💡 Tips
Contextual gardening tips based on user's interest tags and experience level.

⚙️ Settings
Profile edit, notification preferences, units toggle, privacy and data controls.

User Flow — Onboarding to First Recommendation

1

🌇 Splash Screen

User opens the app for the first time. A 3D animated mega-city scene renders with glowing building windows and heat shimmer columns. The scene fades automatically after 2.5 seconds. Returning users with a valid NextAuth session skip onboarding entirely and land directly on the Home Dashboard.

2

👋 Onboarding Carousel

Three illustrated slides explain the HeatWise value proposition: (1) Measure your space, (2) Get AI species picks, (3) Cool the city. A "Get Started" button leads to phone login. A "Skip" option leads straight to profile setup if auth is not required.

3

📱 Phone Authentication

User enters their phone number. The system normalises the number and calls

`POST /api/phone-auth/send-otp` . A 6-digit OTP is generated, hashed, stored with an expiry, and sent via SMS (or logged to console in dev mode when HEATWISE_DEV_OTP=true).

`POST /api/phone-auth/send-otp`

4

🔢 OTP Verification

User enters the 6-digit code within the 10-minute expiry window. The system verifies the hash, checks rate limits, and on success calls NextAuth to create a session. If the user's profile is incomplete, the app redirects to Profile Setup rather than Home.

`POST /api/phone-auth/verify-otp`

5

👤 Profile Setup (4 Steps)

Step 1 — Name and age input.
Step 2 — City: auto-detected via browser Geolocation API + Open-Meteo reverse geocode, or manually typed.
Step 3 — Gardening experience (Beginner / Intermediate / Expert) and interest chips (Food Garden, Aesthetics, Herbs & Spices, Wildlife, Rooftop Farm, Zen Garden, etc.).
Step 4 — Space type (Rooftop / Terrace / Balcony / Semi-outdoor) and estimated size slider (5–500 m²). Saved to localStorage as `hw_profile`.

6

🏠 Home Dashboard

Animated City Cooling Index score (derived from total green area + project count). Four metric cards: Garden Area, Temp Drop, CO_2/yr, Energy Saving — all calculated from live project data fetched from

`GET /api/projects` . Personalized tip based on interest tag. Carbon footprint card. Explore shortcuts. Recent scans list.

`GET /api/projects`

7

＋ Create Project

User taps the FAB (+) or Scan nav item. Enters project name, selects surface type (rooftop/terrace/balcony), and sets primary goal. A project record is created in the database before measurement begins.

`POST /api/projects`

8

📏 Measurement (3 paths)

Path A — Live AR: Device camera streams to canvas. Corner markers placed via tap using perspective-correct ray-floor intersection math (device pitch β + roll γ from DeviceOrientationEvent). Width × length auto-calculated.
Path B — Photo AR: User imports or captures a photo. Gallery-mode corrects for objectFit:contain letterboxing. Angle picker (Overhead / High / Mid / Low) substitutes for live device orientation.
Path C — Manual: Sliders for width (m), length (m), and floor level.

9

🌤️ Environment Detection

Auto mode: browser Geolocation → `POST /api/env/detect` fetches Open-Meteo (temperature, wind speed, UV index) + Nominatim reverse geocode. Derives: heatExposure, windExposure, sunExposure, windLevel. Manual mode: sliders for sun exposure, wind level, summer temperature.

`POST /api/env/detect`

10

🧠 Analysis + Recommendation Generation

A neural-brain animation plays while the server processes. The app calls

`POST /api/recommendations/generate` with the full photo session payload: dimensions, environment signals, project metadata, user location. The engine runs: ML species scoring → catalog hybrid fallback → layout eligibility → rollout guardrails → geo enrichment → pricing. Returns ranked layout slates with candidates.

`POST /api/recommendations/generate`

User Flow — Results, Install & Carbon

11

🌿 Results Screen

Ranked recommendation cards displayed with: species name, image, cooling score badge, estimated cost, sun/wind compatibility chips, and CO_2 impact per year. Three filter chips at the top let the user narrow by sun exposure, wind tolerance, or cooling priority. Each card tap expands to show placement zones and maintenance level.

`POST /api/recommendations/log-impression`

12

🗺️ Garden Layout View

The selected recommendation's layout slate is rendered as a spatial plan: zone polygons for north wall, perimeter, and center fill. Each zone shows species placement count, coverage area, and expected shading. The user can tap zones to see species detail or re-rank alternatives.

13

✨ Before / After Visualization

A 3D Three.js scene renders the space before (bare rooftop) and after (with recommended green cover). Swipe or slider animates between states. An overlay shows the projected temperature delta and CO_2 benefit. Users can share this view as a screenshot.

`POST /api/generate-garden-visual`

14

📄 Report / Dossier

A detailed PDF-style report generated on-demand: species fact sheet, layout zones, estimated installation cost range, material list, watering schedule, and projected 5-year CO_2 benefit. The report can be downloaded or shared with an installer.

`POST /api/reports/recommendation-dossier`

15

🔧 Installer Assignment

User requests installation. The system searches verified installers by city and availability via

`POST /api/installers/search`. The matched installer receives a job notification. The user can request a quote; the installer submits pricing. On acceptance, a job record is created. Status updates (accepted → in-progress → completed) are tracked in real time.

`POST /api/installers/request-quote` `POST /api/installers/accept-job`

16

✅ Install Confirmed + Follow-up

After the installer submits a verified install record, the user sees a confirmation screen with before/after thumbnails and their new carbon offset calculation. A longitudinal follow-up schedule is created (30, 90, 180 days) to collect remeasurement data and improve the ML model.

`POST /api/installers/submit-verified-install` `POST /api/followups/create-schedule`

Carbon Footprint User Journey

A

🌍 Carbon Setup (4 inputs)

User sets: car km/week, flights per year, electricity kWh/month, and diet type (Vegan / Vegetarian / Pescatarian / Omnivore / Heavy Meat). A live total badge in the header updates in real time using India-specific emission factors (car 0.210 kg/km, electricity 0.820 kg/kWh, LPG 2.983 kg/kg). Saved to localStorage as `hw_carbon_profile`.

B

📊 Carbon Dashboard

Period switcher (Week / Month / Year). Main number shows total CO_2 in selected period. Letter grade badge (A+ to D) based on net footprint after garden offset. Breakdown bars for Transport, Energy, Diet, Other. Garden offset row showing area × 0.021 T/yr with offset percentage progress bar. Net after garden row. India average comparison (1.9 T/yr baseline).

City Heat Map Journey

C

🗺️ City Heat Map

User navigates to Explore tab. A holographic 3D globe (Three.js) shows the city with temperature colour-coding overlaid on districts. Live temperature data is fetched via Open-Meteo. Users can tap any district to see: current temperature, heat index classification (low / medium / high / extreme), and the number of HeatWise-monitored green installations in that zone.

D

🌱 Species Library

Searchable and filterable catalog of all species in the recommendation engine's database. Filters: sun exposure, water demand, pet-safe, drought-tolerant, edible, space type compatibility. Each species card links to a detail page with ASPCA safety status, watering frequency, maintenance level, and cooling score range.

Feature Catalogue

Live AR Measurement

Real-time perspective-correct corner placement using DeviceOrientationEvent (beta pitch, gamma roll). Converts screen coordinates to real-world metres via ray-floor intersection math. Supports both live camera and imported photos (gallery mode with letterbox correction). Cross-browser canvas rounded-rect with quadraticCurveTo fallback for older Android WebViews.

CORE SENSOR FUSION

AI Recommendation Engine

Hybrid 3-tier engine: (1) ML species multi-label classifier served via Python runtime, (2) rule-based catalog hybrid fallback with pet-safe / drought-tolerant / water-scarce hard-exclusion logic, (3) rules-only fallback. Outputs ranked layout slates with zone placements. A/B experiment rollout with guardrails and shadow evaluation.

ML CORE

Auto Environment Detection

Browser Geolocation API feeds lat/lon to the server-side env/detect endpoint. Fetches Open-Meteo (temperature_2m, wind_speed_10m, uv_index, daily max temp) and Nominatim reverse geocode. Derives 4 semantic signals: heatExposure, windExposure, sunExposure, windLevel. Falls back gracefully to manual sliders.

CORE

Installer Network

Full marketplace: search, assignment, quote request/submit/compare, job status tracking (accepted → in-progress → verified), outcome verification, SLA monitoring, and commercial performance dashboards. Installers have their own portal with field status, capability profiles, and revenue event logging.

CORE

Carbon Footprint Tracking

Captures 4 lifestyle inputs (car km/week, flights/year, electricity kWh/month, diet). Applies India-specific emission factors. Derives weekly / monthly / annual CO_2 footprint. Calculates garden offset using area × 0.021 T/yr. Letter grade A+ to D. Net vs India average (1.9 T/yr) comparison.

CORE

3D Before/After Visualisation

Three.js scene renders the rooftop space in before (bare) and after (planted) states. Shareable screenshot. Temperature delta overlay and CO_2 benefit badge. Exportable via installer export builder for client presentations.

3D NATIVE

City Heat Map

Holographic Three.js globe overlaid with live temperature data by district. Real-time heat zone

Species Library & Catalog

Full catalog backed by PostgreSQL SpeciesCatalog model. Filterable by sun, water, drought tolerance,

classification. Shows density of HeatWise-monitored green installations. Helps users contextualise their contribution to the city-level heat island mitigation effort.

3D

pet safety, edibility, space type. Generated mapping artifact (species_catalog_mapping.v1.json) committed to repo for CI validation. ASPCA toxicity data and moisture requirements embedded in seed data.

CORE ML DATA

📱

Phone OTP Authentication

Passwordless authentication via SMS OTP. Number normalisation, hash storage, expiry management, rate limiting. NextAuth session management. Dev mode logs OTP to console instead of SMS. Protected route middleware checks session status on every navigation.

AUTH

📊

Admin Analytics Dashboard

Real-time metrics: recommendation run counts, average selected rank, install conversion rate, save rate, visualisation rate, regeneration rate, heat reduction delta, MAU. Segment analysis by location, budget, experience. Installer commercial outcomes, cohort retention, and LTV tracking.

ADMIN

🔬

ML Training Pipeline

Telemetry export → scikit-learn training (species multi-label classifier + ranking model) → model registry versioning → Python serving endpoint. Automated retraining workflow. Shadow evaluation compares live model vs challenger model on every recommendation run. Verified install outcomes feed back as ground truth.

ML

🏗️

Android Native App

Capacitor wrapper around the Next.js PWA. WebView points at CAP_SERVER_URL. Supports USB adb reverse (localhost:3000), Android emulator (10.0.2.2:3000), and LAN Wi-Fi. Network security config allows HTTP cleartext in dev. One-command sync + Android Studio open workflow.

ANDROID

📅

Longitudinal Follow-up System

After verified installation, schedules follow-up check-ins at 30, 90, and 180 days. Captures remeasurement data (plant survival, cooling delta, maintenance rating). Data feeds back into the ML training pipeline as verified outcome labels to improve future recommendation quality.

CORE ML DATA

🔒

Governance & Compliance

GDPR-ready consent management, data retention policies, policy flag system, audit event log. User consent records stored with timestamp and scope. Governance review summary endpoint for regulatory reporting. Platform readiness health check endpoint for uptime monitoring.

COMPLIANCE

💰

Commercial & Pricing Engine

Cost estimation per recommendation (species + labour + materials). Budget fit assessment. Pricing preview before commitment. Quote comparison across multiple installers. Revenue event logging, funnel conversion tracking, cohort analysis, and unit economics dashboard for business intelligence.

📡

Supply Chain Integration

Real-time species and material availability checks per region. Seasonal planting window validation (prevents recommending out-of-season species). Regional readiness assessment. Supply constraint preview shown to user before finalising

BUSINESS

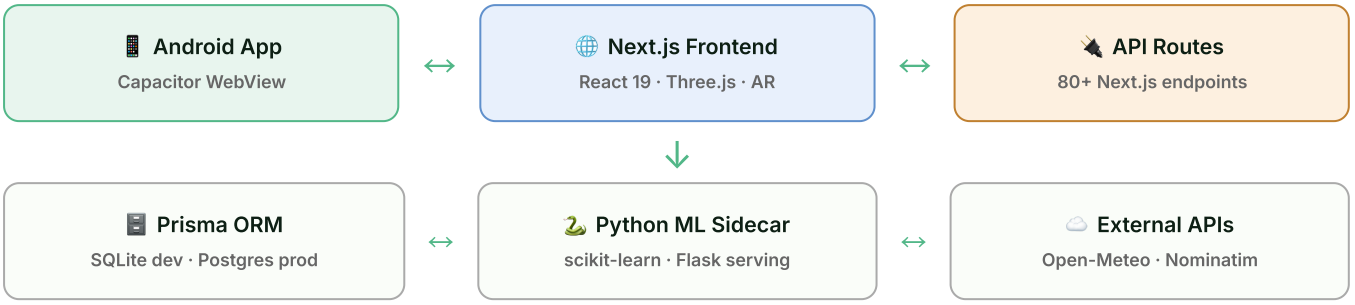
recommendation. Webhook integration for partner inventory systems.

SUPPLY

System Architecture Overview

Full-Stack Monorepo on Next.js Pages Router

The entire platform lives in a single Next.js repository. API routes, frontend components, ML integration, and Capacitor Android wrapper are co-located. Prisma provides type-safe database access. The Python ML serving process is a sidecar that Next.js calls via HTTP on localhost.



Frontend Stack

`Framework`	Next.js 16 with Pages Router. All pages live in `/pages/`. Main app component is a large screen-switch in `components/HeatWiseApp.jsx`.
`3D / AR`	Three.js for all 3D scenes (mega city, holo globe, neural brain, garden before/after). WebGL canvas overlays for AR measurement. Custom perspective math using DeviceOrientationEvent.
`State`	Local React state + localStorage for profile/carbon. No global state library. Data fetched per-screen with useEffect + fetch().
`Auth`	NextAuth with custom phone-OTP provider. SessionProvider wraps app in _app.jsx. useSession() used on protected screens.
`Styling`	Inline React styles throughout with design tokens in `components/heatwise/theme.js`. DM Sans + Space Grotesk + JetBrains Mono fonts. CSS animations via injected <style> tags.
`Path Aliases`	@/* → root, @/models → ./models, @/ar → ./src/ar, @/live-ar/* → ./src/live-ar/*. Configured in jsconfig.json and tsconfig.json.

Backend & Database

Runtime	Next.js API routes (TypeScript). Each route is a serverless function. No separate Express/Fastify server needed.
ORM	Prisma 5.x. Schema in prisma/schema.prisma. 55+ models. Dev uses SQLite (file:./dev.db). Production uses PostgreSQL via DATABASE_URL.
Seed Data	prisma/data/species_catalog_seed.mjs seeds SpeciesCatalog. Separate alias extensions in species_alias_extensions.json. Generated mapping artifact must be committed.
Auth Storage	PhoneOtp table stores hashed OTPs with expiry. User table stores phone, name, profileCompleted flag, city, gardening score. UserSettings and UserConsentRecord for preferences and compliance.

Directory Structure

```
/heatwise ├── components/ # React components (HeatWiseApp.jsx + sub-components) ├── pages/ # Next.js
pages + 80+ API routes │ └── api/ # recommendations/, installers/, admin/, env/, ... ├── src/ # Live AR
screens, tracking, geometry math │ └── live-ar/ # LiveARMeasurementScreen.tsx + adapters ├── lib/ #
Recommendation engine, ML, services, auth │ └── recommendation/ # orchestrate, fallback, eligibility,
rollout ├── ml/ # Python serving: training, inference, evaluation ├── ml-models/ # Standalone model
training scripts ├── heatwise-native/ # Capacitor Android shell ├── prisma/ # Schema, migrations, seeds
├── data/species/ # species_catalog_mapping.v1.json (generated) └── models/ # Domain type definitions
(index.ts)
```

Recommendation Engine — Deep Dive

Engine Architecture (3-Tier Hybrid)

Design Principle

The engine always returns a result. If the ML model is unavailable, the catalog hybrid fallback fires. If the catalog is incomplete, the rules-only fallback guarantees at least 3 ranked candidates. Every tier produces the same output shape.

```
// Full recommendation pipeline — lib/recommendation/orchestrateLayoutRecommendations.ts POST
/api/recommendations/generate │ ├─ buildRecommendationGenerateRequestFromPhotoSession() │ photoSession →
ProjectInput → RecommendationGenerateRequest │ ├─ enrichRecommendationsWithPricing() // cost per
candidate ├─ buildGeoEnrichmentBundle() // lat/lon → microclimate │ ├─ // TIER 1: ML (Python sidecar via
HTTP) │ mlRecommendationService.predict() │ → species scores (multi-label classifier) │ → ranking model
scores │ ├─ // TIER 2: Catalog Hybrid Fallback (if ML unavailable) │ catalogHybridFallback() │ →
HARD_PET_UNSAFE_SPECIES exclusion │ → waterScarcityHardExclude (drought-tolerant check) │ →
petSafeRankingNudge / reconcileChildPetSafety │ → score candidates by environment match │ ├─ // TIER 3:
Rules-Only Fallback │ rulesOnlyFallback() // guaranteed 3+ results │ ├─ layoutEligibility() // north wall
/ perimeter / center ├─ mergeMlIntoLayoutRecommendations() // combine scores + layout ├─
recommendationRolloutGuardrails() // A/B experiment safety ├─ attachGeoExplanationToCandidates() // "why
this species" └─ scheduleRecommendationRuntimeObservation() // async telemetry
```

Species Scoring Logic

Each species is scored on a weighted combination of environment match signals. The catalog hybrid fallback uses these rules:

Sun Exposure Match

Compares species sunRequirement (shade / partial / full) against the derived sunExposure from UV index or heat exposure. Full sun species are penalised in shaded environments and vice versa.

Wind Tolerance

windExposure (sheltered / moderate / windy / severe) mapped to windLevel (low / medium / high). Species with low wind tolerance are excluded when windExposure ≥ windy.

Water Scarcity Hard Exclusion

If scenario has scarce water: species with droughtTolerant=false AND unknown effective water demand are hard-excluded. Prevents recommending water-hungry plants in arid contexts.

Pet Safety Hard Exclusion

If child_pet_safe_required=1: species with dbPetSafe=false are hard-excluded before ranking. ASPCA toxicity data embedded in seed. Lemongrass and mint are correctly flagged as pet-unsafe (ASPCA-confirmed).

Space Type Filtering

spaceType (rooftop / terrace / balcony) filters out species with incompatible height, root depth, or weight requirements. Balcony scenarios exclude large-root species automatically.

Layout Zone Eligibility

layoutEligibility.ts assigns each species to valid placement zones: north wall (tall climbers), perimeter (medium shrubs), center fill (ground cover). Returns eligibility codes per zone with failure reasons.

A/B Experiment & Rollout System

`Assignment`	POST /api/recommendations/assignment returns experiment variant (control / challenger) per user session.
`Guardrails`	recommendationRolloutGuardrails.ts checks: minimum species count, maximum exclusion rate, diversity score floor. Rolls back to control if challenger fails guardrails.
`Shadow Eval`	Every recommendation run optionally runs appendRuntimeEvaluation() which scores the challenger variant without serving it to the user. Results stored in RecommendationRuntimeObservation.
`Telemetry`	RecommendationTelemetrySession (immutable per request) + RecommendationCandidateSnapshot (exact card shown) + RecommendationTelemetryEvent (user signals). All append-only — never updated.

ML Pipeline & AR System

ML Model Architecture

`Classifier`	Species multi-label classifier (scikit-learn). Input features: spaceType, sunExposure, windLevel, floorLevel, budgetRange, maintenanceLevel, lat/lon. Output: probability scores for each species code in the catalog.
`Ranker`	Ranking model trained on telemetry: which species the user selected given the ranked list. Optimised for NDCG. Outputs a re-ranking score combined with classifier probability.
`Serving`	Python Flask/FastAPI sidecar in /ml/serving/. TypeScript calls it via HTTP using lib/services/mlRecommendationService.ts. Config via env vars: HEATWISE_ML_CWD, HEATWISE_REGISTRY_DIR, HEATWISE_SPECIES_CSV.
`Registry`	ml/registry/ stores versioned model artefacts. Each version includes: model weights, feature schema, evaluation metrics, training dataset hash. Enables safe rollback.

Training Data Pipeline

1. Telemetry Collection

Every recommendation run creates a RecommendationTelemetrySession. User selections, impressions, and feedback logged as RecommendationTelemetryEvent rows. All immutable and append-only.

2. Outcome Verification

Installed projects produce VerifiedInstallRecord after installer submits photos. Follow-up check-ins at 30/90/180 days provide survival and cooling delta data as ground truth labels.

3. Export Pipeline

npm run ml:export-telemetry-pipeline exports labelled training data from Prisma to CSV files in ml/data/. Feature engineering applied: derive heat level from temp, encode categoricals.

4. Model Training

ml-models/scripts/train_species_model.py runs scikit-learn training. Validated against held-out test set. Metrics (precision@k, NDCG, recall) logged to registry.

5. Deploy & Shadow Eval

New model deployed to registry. Rollout guardrails activate shadow evaluation: new model runs alongside control without affecting user output. If guardrails pass over N requests, traffic shifted to new model.

AR Measurement System

`File`	src/live-ar/screens/LiveARMeasurementScreen.tsx — primary AR screen.
`Perspective Math`	screenToWorld() uses full ray-floor intersection. Given a tap (sx, sy) in screen space, device pitch β and roll γ, image width W and height H, derives a unit direction vector and intersects with y=0 floor plane to get real-world (x, z) coordinates.

Live Mode	DeviceOrientationEvent provides beta (pitch) and gamma (roll) in real time. Tapping places corner markers. After 4 corners, area = |cross product| / 2 × 2 for rectangle approximation.
Gallery Mode	Static photo has no device orientation. getContainedImageBounds() computes objectFit:contain letterbox offset from naturalWidth/Height vs clientWidth/Height. galleryBeta state (default 60°) substitutes for live pitch. 4 preset buttons: Overhead (β=20), High (β=42), Mid (β=60), Low (β=76).
Canvas Compat	rRect() helper: uses native ctx.roundRect() if available (Chrome 99+, Safari 15.4+, Firefox 112+), falls back to quadraticCurveTo() for older Android WebViews. All 8 call sites use this helper.
Overlay	Canvas drawn on top of camera/image. Corner markers, edge lines, area label, tip banner, mini-map all drawn with requestAnimationFrame loop. DM Sans font loaded for labels.

07 API Reference — Core Endpoints

Authentication

Method	Endpoint	Description
POST	/api/phone-auth/send-otp	Normalise phone number, generate + hash OTP, send SMS (or console log in dev)
POST	/api/phone-auth/verify-otp	Verify hashed OTP against stored value. On success, create NextAuth session
POST	/api/auth/[...nextauth]	NextAuth catch-all: session management, callbacks, JWT handling

User & Profile

Method	Endpoint	Description
GET	/api/user/me	Return authenticated user profile (name, city, gardening score, profileCompleted)
POST	/api/user/profile	Update user profile fields. Sets profileCompleted=true on first save
POST	/api/user/settings	Update notification preferences, units (metric/imperial), theme

Projects

Method	Endpoint	Description
GET	/api/projects	List all projects for authenticated user with area, status, last modified
POST	/api/projects	Create project (name, surfaceType, primaryGoal). Returns projectId for session linking
GET	/api/projects/[id]	Fetch single project with full analysis and recommendation history
POST	/api/projects/[id]	Update project metadata, mark as draft/active/archived

Recommendation Engine

Method	Endpoint	Description
POST	/api/recommendations/generate	**Main endpoint.** Accepts full photo session payload. Runs ML + fallback chain + layout + pricing. Returns ranked candidates with layout slates
POST	/api/recommendations/create-session	Persist recommendation session to DB. Called after generate returns results
POST	/api/recommendations/assignment	Return experiment variant (control/challenger) for this user+session
POST	/api/recommendations/mark-selected	Log which recommendation the user tapped. Updates rank analytics
POST	/api/recommendations/log-impression	Log that a recommendation card was displayed (for impression-level analytics)

POST	/api/recommendations/log-interaction	Log detail expansions, filter changes, saves, share events
POST	/api/recommendations/submit-feedback	Accept user rating (1–5) and optional text feedback on recommendation quality
POST	/api/recommendations/explain	Return natural-language explanation of why a species was recommended
POST	/api/recommendations/evaluate	Run quality evaluation for a recommendation run (for admin review)
POST	/api/recommendations/rollout-status	Check current rollout percentages for active experiments
POST	/api/recommendations/submit-install-outcome	Log verified installation as positive/negative outcome label for ML retraining

Environment & Geospatial

Method	Endpoint	Description
POST	/api/env/detect	Accepts {lat, lon}. Fetches Open-Meteo + Nominatim. Returns heatExposure, windExposure, sunExposure, windLevel, locationLabel
POST	/api/geospatial/site-exposure	Compute wind and sun exposure scores for a specific lat/lon site
POST	/api/geospatial/microclimate	Fetch microclimate snapshot (urban heat island factor, albedo, green fraction)
POST	/api/geospatial/enrich-project	Attach geo context bundle to an existing project record

Garden Visualisation

Method	Endpoint	Description
POST	/api/generate-layout	Legacy layout generation endpoint (ProjectInput + linkage). Returns zone polygons
POST	/api/generate-garden-visual	Generate 3D before/after garden visual parameters for Three.js scene

API Reference — Installer, Admin & Supply

Installer Network

Method	Endpoint	Description
POST	/api/installers/search	Find verified installers by city, availability, and service type
POST	/api/installers/assign	Assign installer to a job. Creates InstallerInstallJob record
POST	/api/installers/request-quote	User requests a price quote from a specific installer
POST	/api/installers/submit-quote	Installer submits pricing quote for a job
POST	/api/pricing/compare-quote	Compare quotes from multiple installers side-by-side
POST	/api/installers/accept-job	Installer accepts an assigned job. Updates status to in-progress
POST	/api/installers/reject-job	Installer rejects job. Triggers reassignment to next eligible installer
POST	/api/installers/update-job-status	Update job to: scheduled / in-progress / completed / cancelled
POST	/api/installers/submit-verified-install	Installer submits photo proof + measurements for outcome verification
POST	/api/installers/submit-outcome-verification	Platform verifies install quality. Creates VerifiedInstallRecord

Pricing & Cost

Method	Endpoint	Description
POST	/api/pricing/estimate	Estimate total cost (species + materials + labour) for a recommendation
POST	/api/pricing/budget-fit	Assess whether a recommendation fits within the user's stated budget range
POST	/api/pricing/pricing-preview	Show itemised pricing before user commits to installation request

Supply Chain

Method	Endpoint	Description
POST	/api/supply/species-availability	Check if recommended species are in stock in the user's region
POST	/api/supply/material-availability	Check planters, growing medium, and structural materials availability
POST	/api/supply/seasonal-windows	Return optimal planting months for each species in the user's climate zone
POST	/api/supply/regional-readiness	Combined check: are species + materials + installer available this month?

Admin Analytics

Method	Endpoint	Description
GET	/api/admin/analytics	Full analytics dashboard: runs, conversion, saves, heat reduction, MAU
GET	/api/admin/metrics-overview	KPI summary cards for quick health check
GET	/api/admin/recommendation-funnel	Funnel from generate → impression → selection → install request → verified
GET	/api/admin/experiment-summary	A/B experiment results: control vs challenger click-through, install rate, satisfaction
GET	/api/admin/rollout-monitor	Real-time rollout percentages and guardrail violation counts
GET	/api/admin/ranking-dataset	Export training dataset for offline analysis
GET	/api/admin/installer-outcomes	Installer performance: completion rate, SLA adherence, customer rating

Commercial & Follow-ups

Method	Endpoint	Description
POST	/api/commercial/log-revenue-event	Log a revenue transaction (installation fee, subscription, tip)
GET	/api/commercial/funnel-summary	Conversion funnel from lead to paid install
GET	/api/commercial/unit-economics	CAC, LTV, payback period, gross margin per cohort
POST	/api/followups/create-schedule	Schedule 30/90/180-day follow-up check-ins after verified install
GET	/api/followups/due-followups	List follow-ups due today for the notification system
POST	/api/followups/submit-remeasurement	User submits plant survival data and cooling observations at follow-up

Governance

Method	Endpoint	Description
POST	/api/governance/consent	Record user consent for data collection, analytics, marketing
GET	/api/governance/review-summary	Compliance review summary for regulatory reporting
GET	/api/platform/audit-events	Append-only audit log of all sensitive operations

Database Models (55+ Prisma Models)

Core Domain Models

Model	Purpose	Key Fields
`User`	Platform user account	phone, name, city, age, gardeningScore, profileCompleted
`PhoneOtp`	OTP storage for SMS auth	phoneNormalized, hash, expiresAt, usedAt, attempts
`UserSettings`	Per-user preferences	units, notificationsEnabled, theme, language
`Project`	Garden project	name, surfaceType, primaryGoal, area, status, userId
`PhotoSession`	Measurement session	widthM, lengthM, floorLevel, captureMode, projectId
`Analysis`	Project analysis result	recommendationRunId, tempDropC, co2yr, energySavingPct
`SpeciesCatalog`	Plant species registry	code, name, scientificName, petSafe, droughtTolerant, sunRequirement, waterDemand, coolingScore

Recommendation & Telemetry Models

Model	Purpose	Key Fields
`RecommendationRun`	Single recommendation generation event	projectId, userId, modelVersion, rulesVersion, duration
`RecommendationTelemetrySession`	Immutable snapshot per request	sessionId, modelVersion, rulesVersion, experimentVariant
`RecommendationCandidateSnapshot`	Exact card shown to user	rank, speciesCode, score, cost, layoutZone
`RecommendationTelemetryEvent`	Append-only user/installer signals	eventType (impression/select/feedback), candidateId, value
`RecommendationRuntimeObservation`	Rollout metrics and guardrail results	variant, guardRailsPassed, diversityScore, exclusionRate

Installer & Commercial Models

Model	Purpose	Key Fields
`InstallationRequest`	User's installation service request	projectId, recommendationId, budgetRange, urgency
`InstallerQuoteRequest`	Quote sent to installer	installerId, installationRequestId, status, sentAt
`InstallerInstallJob`	Active installation job	installerId, jobStatus, scheduledDate, completedAt
`VerifiedInstallRecord`	Confirmed installation outcome	photoUrls, measuredAreaM2, plantSurvivalPct, coolingDeltaC
`RevenueEvent`	Commercial transaction	type, amount, currency, installJobId, userId
`LeadFunnelEvent`	Funnel conversion tracking	stage (generate/select/request/install), userId, timestamp
`CommercialOutcome`	Aggregated commercial result	cohort, CAC, LTV, paybackDays, grossMarginPct

Geo, Follow-up & Compliance Models

Model	Purpose	Key Fields
`GeoContext`	Geographic context per project	latitude, longitude, locationLabel, heatExposure, windExposure
`MicroclimateSnapshot`	Urban heat island data	heatIslandFactor, albedo, greenFraction, fetchedAt
`CostEstimateSnapshot`	Pricing snapshot per recommendation	speciesCost, labourCost, materialCost, total, currency
`LongitudinalFollowupSchedule`	Follow-up plan after install	dueAt (30/90/180d), status, remeasurementId
`FollowupRemeasurement`	User-reported outcome at follow-up	plantSurvivalPct, coolingSatisfaction, maintenanceRating
`UserConsentRecord`	GDPR consent tracking	scope, grantedAt, revokedAt, ipAddress
`GovernancePolicyFlag`	Compliance policy violations	flagType, severity, resolvedAt, reviewedBy
`AuditEvent`	Append-only sensitive op log	operation, entityType, entityId, actorId, timestamp

Telemetry Immutability Guarantee

All telemetry models (RecommendationTelemetrySession, RecommendationCandidateSnapshot, RecommendationTelemetryEvent, RecommendationRuntimeObservation) are append-only. Once written, rows are never updated or deleted. This ensures auditability and reliable ML training data.

Environment, Deployment & CI

Environment Variables

```
# Database DATABASE_URL="file:./dev.db" # SQLite (dev) | postgres:// ... (prod) # Auth
NEXTAUTH_URL=http://localhost:3000 NEXTAUTH_SECRET=your-secret-here HEATWISE_DEV_OTP=false # true → log
OTP to console (no SMS) HEATWISE_ADMIN_EMAILS=admin@example.com # AI OPENAI_API_KEY=sk-... # ML Serving
(Python sidecar) HEATWISE_ML_CWD=./ml HEATWISE_REGISTRY_DIR=./ml/registry
HEATWISE_SPECIES_CSV=./ml/data/bootstrap/sample_outputs/demo_pack/species_features.csv # Android /
Capacitor CAP_SERVER_URL=http://localhost:3000 # WebView target URL
HEATWISE_DEV_EXTRA_ORIGINS=http://192.168.1.X:3000 # LAN Wi-Fi physical device
```

Development Commands

Command	Purpose
`npm run dev`	Start Next.js dev server on port 3000
`npm run lint`	ESLint check across all source files
`npm run typecheck`	TypeScript validation (tsc --noEmit)
`npm run db:seed`	Seed the database (species catalog + demo data)
`npm run db:migrate`	Run pending Prisma migrations
`npm run gen:species-mapping`	Regenerate data/species/species_catalog_mapping.v1.json (must commit result)
`npm run check:species-mapping`	Verify mapping artifact is up to date (runs in CI)
`npm run test:recommendation`	Integration tests for recommendation engine (requires DATABASE_URL + seeded DB)
`npm run ml:serving-smoke`	Smoke test the Python ML serving endpoint
`npm run dev:android`	Start Next.js for Android (0.0.0.0 bind + adb reverse)
`npm run android:sync`	Install deps + run cap sync android
`npm run android:studio`	Sync + open Android Studio in one command

CI Pipeline (GitHub Actions)

`Trigger`	Every push and pull request to main branch.
`Step 1`	npm run check:species-mapping — verifies the generated species catalog mapping JSON is committed and matches current seed data. Fails if stale.
`Step 2`	npm run typecheck — full TypeScript validation. Fails on any type error across all .ts and .tsx files.

Android Deployment Targets

Target	CAP_SERVER_URL	Setup
Android Emulator	`http://10.0.2.2:3000`	Special emulator loopback. No adb reverse needed.
USB Physical Device	`http://localhost:3000`	Run: adb reverse tcp:3000 tcp:3000 before launching
Wi-Fi Physical Device	`http://192.168.1.X:3000`	Set LAN IP in CAP_SERVER_URL and NEXTAUTH_URL. Add to HEATWISE_DEV_EXTRA_ORIGINS.

Production Deployment

Web Hosting Vercel (Next.js first-class support). Environment variables set in Vercel dashboard. DATABASE_URL points to managed PostgreSQL (Vercel Postgres or Supabase).

Database PostgreSQL in production. Migrations via npx prisma migrate deploy in CI/CD pipeline before new version deploy.

ML Serving Python sidecar deployed as a separate service (Cloud Run or Railway). HEATWISE_ML_CWD and HEATWISE_REGISTRY_DIR point to cloud storage paths.

Android Build npm run build → cap sync android → Android Studio release build → signed APK/AAB → Google Play Store. CAP_SERVER_URL points to production Vercel URL.

CORS Next.js allows 10.0.2.2 (emulator loopback) and localhost by default in dev. Production origins locked to Vercel URL. Additional origins via HEATWISE_DEV_EXTRA_ORIGINS.

Key Architectural Decisions

Monorepo: All code in one repo reduces integration friction between frontend, API, and ML.
Pages Router: Chosen over App Router for stability with NextAuth and the large existing component tree.
Inline styles: Eliminates CSS class name collisions in the single-file component architecture of HeatWiseApp.jsx.
Append-only telemetry: Prevents data loss and enables reliable ML retraining on immutable historical data.
3-tier recommendation fallback: Guarantees a result is always returned, even when ML serving is unavailable.